

19-08-2021

# GREY BOX WEB APPLICATION SECURITY AUDIT REPORT OF CHILIKA DEVELOPMENT AUTHORITY APPLICATION

Version 1.0

## Abstract

This document details the grey box web application security audit activity conducted from the internet on Chilika Development Authority Application

## Author

IS Consulting Group, SecurEyes  
India | UAE | Oman | US | KSA

### **Document Information**

|                                |                                                                                               |
|--------------------------------|-----------------------------------------------------------------------------------------------|
| <b>Document Title</b>          | Grey Box Web Application Security Audit Report of “Chilika Development Authority Application” |
| <b>Document Classification</b> | Confidential, Internal to SecurEyes and “STLINDIA”                                            |
| <b>Document Type</b>           | Audit Report                                                                                  |

### **Document History**

| # | Date                            | Version | Description        | Author         | Reviewer                     | Approval |
|---|---------------------------------|---------|--------------------|----------------|------------------------------|----------|
| 1 | 10 <sup>th</sup> August<br>2021 | 1.0     | Initial<br>Version | Abhijeet Panda | Susree<br>Swagatika<br>Sahoo | Ujal Ray |

This document and any revised pages are subject to document control. Please keep them up to date using the release notices from the distributor of the document.

## Table of Contents

|     |                                                            |                                     |
|-----|------------------------------------------------------------|-------------------------------------|
| 1.  | Executive Summary.....                                     | 3                                   |
| 1.1 | Introduction .....                                         | 3                                   |
| 1.2 | Scope.....                                                 | 3                                   |
| 1.3 | Summary of Findings.....                                   | 4                                   |
| 1.4 | Audit Constraints: .....                                   | 5                                   |
| 1.5 | Application Vulnerabilities Discovered .....               | 6                                   |
| 1.6 | OWASP Finding Summary .....                                | 8                                   |
| 1.7 | Technical Findings .....                                   | 9                                   |
| 2.  | Key Findings & Action Items .....                          | 10                                  |
| 3.  | Appendix – 1: Recommended Action Plan.....                 | 26                                  |
| 4.  | Appendix – 2: Guidelines for Developers.....               | 27                                  |
| 4.1 | Validate Input and Output .....                            | <b>Error! Bookmark not defined.</b> |
| 4.2 | Challenge Response Mechanism .....                         | <b>Error! Bookmark not defined.</b> |
| 4.3 | Salted SHA256 technique for the authentication module..... | 27                                  |
| 5.  | Appendix – 3: Interpretation of Ratings .....              | 28                                  |
| 6.  | Appendix – 4: Tools Used.....                              | 29                                  |

## 1. Executive Summary

### 1.1 Introduction

SecurEyes performed a Grey Box Web Application Security Audit of Chilika Development Authority Application. The objective of the audit was to find out vulnerabilities that can be seen and compromised by malicious users over internet.

This report documents the results of the Grey Box Web Application Security Audit activity. The activity was carried out from the source IP Address **115.242.196.222**. The application was found to be vulnerable to multiple security risks including **Insecure Transmission of Password, Insecure Cookie Attributes** etc. The following sections provide details of the scope of the application security audit and the detailed findings from the audit exercise.

Our opinion provided in this report is valid for the period during which the audit was carried out and is based on the information and the application contents available for audit. Projection of any conclusions based on our findings for future periods and application versions is subject to the risk that the validity of such conclusions may be altered because of changes made to the network or application or system or the failure to make the changes to the network, application when required.

### 1.2 Scope

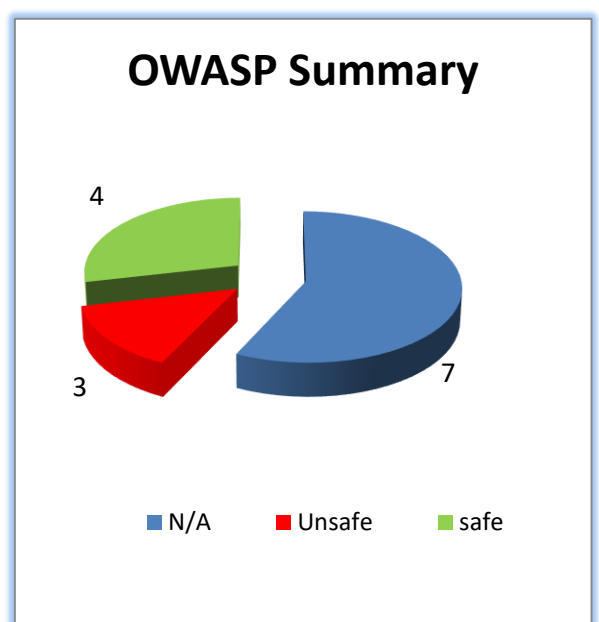
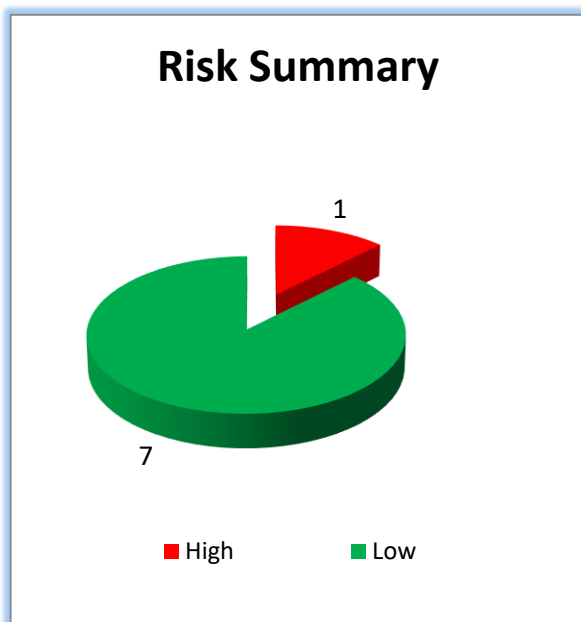
The following table provides an overview on the scope of the grey box web application security audit.

|                         | <i>Details</i>                                                                              |
|-------------------------|---------------------------------------------------------------------------------------------|
| <b>Application Name</b> | Chilika Development Authority Web Application                                               |
| <b>Audit URL</b>        | <a href="http://sgov.stlindia.com/staging1/CDA/">http://sgov.stlindia.com/staging1/CDA/</a> |
| <b>Mode of Test</b>     | Manual                                                                                      |
| <b>Testing Type</b>     | Grey Box                                                                                    |
| <b>Duration</b>         | 28 <sup>th</sup> July 2021 – 10 <sup>th</sup> August 2021                                   |

### 1.3 Summary of Findings

We have identified 01 High and 07 Low Risk vulnerabilities during the grey box web application security audit activity. The following table & graph depicts the number of observed vulnerabilities across risk levels.

| <i>No of Vulnerabilities</i> | <i>High</i> | <i>Medium</i> | <i>Low</i> |
|------------------------------|-------------|---------------|------------|
| 08                           | 01          | 00            | 07         |



#### 1.4 Audit Constraints:

| <i>Module Name</i>     | <i>Details</i>                                                                                 |
|------------------------|------------------------------------------------------------------------------------------------|
| User Registration page | A user is unable to register in this application as Registration functionality is not working. |
| Login Page             | A user is unable to login this application as no login credentials are given.                  |
| Forgot Password        | A user is unable reset the password as forgot password functionality is not working.           |

## 1.5 Application Vulnerabilities Discovered

Following is the list of vulnerabilities observed during the grey box web application security audit activity.

| #  | Observation and Impact                                                                                                                                                                                                     | Technical Finding Name                     | Risk Rating |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------|-------------|
| 1. | The application transmits clear text password from login page over HTTP from client to server. An attacker may sniff the clear text password from the network and later compromise the account.                            | Insecure Transmission of Password          | High        |
| 2. | The application is using vulnerable version of framework in the client-side, by which an attacker can possibly exploit already known vulnerabilities.                                                                      | Using Component with known vulnerabilities | Low         |
| 3. | The application has not implemented 'Domain' cookie attribute. The application cookie may get compromised by the attacker if these cookie attributes are not set properly for the application.                             | Insecure Cookie Attributes                 | Low         |
| 4. | The application has enabled copy and paste feature on the password field of 'Login page' page as a result an malicious user can steal the password as copy paste option is enabled thus compromising the victim's account. | Clipboard Data Stealing Attack             | Low         |

| #  | Observation and Impact                                                                                                                                                                        | Technical Finding Name | Risk Rating |
|----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|-------------|
| 5. | The application has enabled the Autocomplete feature. An attacker can take advantage of the Autocomplete browser feature and can steal username, passwords of valid users of the application. | Autocomplete Issue     | Low         |
| 6. | An attacker can launch clickjacking attack against application user and can trick an application user to perform different activities allowed in the application without his/her consent.     | Clickjacking Attack    | Low         |
| 7. | The application is stores the static MD5 key value in text format in page source which may allow an attacker carry out specific targeted attacks against the application.                     | Static md5 Key Used    | Low         |
| 8. | The captcha is not properly implemented in the application: a) CAPTCHA is travelling in clear text format in response                                                                         | CAPTCHA In response    | Low         |



## 1.6 OWASP Finding Summary

SecurEyes web application security audit is modelled along the methodologies specified by the Open Web Applications Security Project (OWASP). OWASP has rated the Top Ten Vulnerabilities found in web applications worldwide. The table shows how the application compares with respect to the OWASP Top 10 2017 list including additional 4 vulnerabilities as listed in previous OWASP top 10 2004, 2010 & 2013. The below status is pertaining to the test cases as applicable against a grey box web application security auditing activity.

| #   | Vulnerabilities                                 | Status |
|-----|-------------------------------------------------|--------|
| 1.  | A1-Injection                                    | Safe   |
| 2.  | A2-Broken Authentication                        | Unsafe |
| 3.  | A3- Sensitive Data Exposure                     | NA     |
| 4.  | A4- XML External Entities                       | NA     |
| 5.  | A5- Broken Access Control                       | NA     |
| 6.  | A6- Security Misconfiguration                   | Unsafe |
| 7.  | A7- Cross Site Scripting                        | Safe   |
| 8.  | A8- Insecure Deserialization                    | NA     |
| 9.  | A9- Using Components with Known Vulnerabilities | Unsafe |
| 10. | A10- Insufficient Logging and Monitoring        | NA     |
| 11. | Unvalidated Input (2004)                        | Safe   |
| 12. | Denial of Service (2004)                        | Safe   |
| 13. | Malicious File Execution                        | NA     |
| 14. | Cross Site Request Forgery                      | NA     |

## 1.7 Technical Findings

As a result of the comprehensive web application security audits conducted on the '**Chilika Development Authority Web Application**', the following technical vulnerabilities were discovered.

1. Insecure Transmission of Password
2. Using Component with known vulnerabilities
3. Insecure Cookie Attributes
4. Clipboard Data Stealing Attack
5. Autocomplete Issue
6. Clickjacking Attack
7. Static md5 Key Used
8. CAPTCHA In response

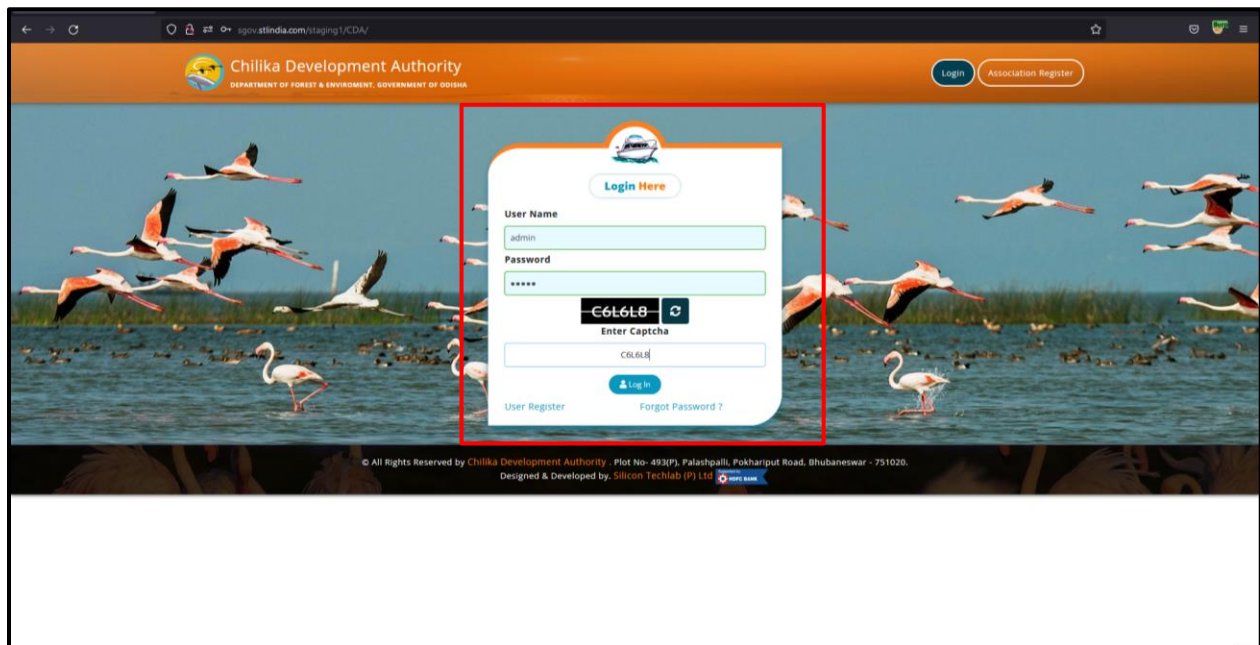
## 2. Key Findings & Action Items

1. The application is transmitting login credentials in cleartext format – Insecure Transmission of Password.

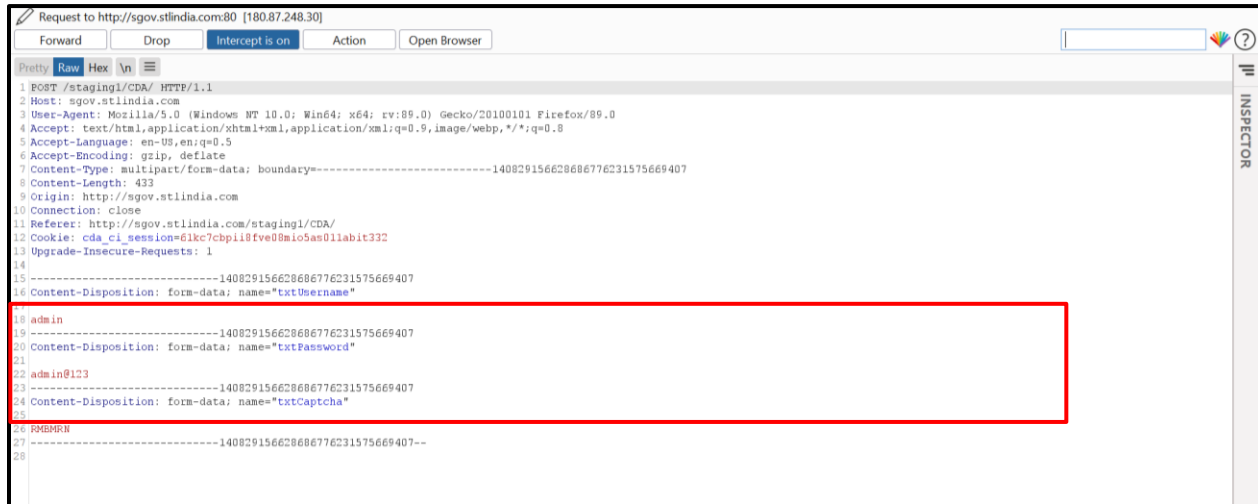
| Observation Details and Impact                                                                                                                                                                  | Technical Finding Name            | Risk Rating | Impact on Application | Probability of Attack |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------|-------------|-----------------------|-----------------------|
| The application transmits clear text password from login page over HTTP from client to server. An attacker may sniff the clear text password from the network and later compromise the account. | Insecure Transmission of Password | High        | High                  | High                  |

### How to test:

**Step #1:** A malicious user navigates to the login page of the application and give random credential as shown below:



**Step #2:** By intercepting the traffic it was observed that the password travel in clear text format as shown below:



```
Request to http://sgov.stilindia.com:80 [180.87.248.30]
Forward Drop Intercept is on Action Open Browser
Pretty Raw Hex \n
1 POST /staging1/CDA/ HTTP/1.1
2 Host: sgov.stilindia.com
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:89.0) Gecko/20100101 Firefox/89.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate
7 Content-Type: multipart/form-data; boundary=-----140829156628686776231575669407
8 Content-Length: 433
9 Origin: http://sgov.stilindia.com
10 Connection: close
11 Referer: http://sgov.stilindia.com/staging1/CDA/
12 Cookie: cda_ci_session=61kc7cbpi18fve08mo5as01labit332
13 Upgrade-Insecure-Requests: 1
14
15 -----140829156628686776231575669407
16 Content-Disposition: form-data; name="txtUsername"
17 admin
18 -----140829156628686776231575669407
19 Content-Disposition: form-data; name="txtPassword"
20 admin@123
21 -----140829156628686776231575669407
22 Content-Disposition: form-data; name="txtCaptcha"
23
24
25
26
27
28
```

**Note:** It is recommended to resolve this issue across all pages of the application where the passwords are transmitted.

### Recommendations:

➤ It is recommended to implement the following:

#### **'Login' Page:**

➤ The application should implement SHA2 salted hashing technique for the password filed.

Please refer '*Salted SHA256 technique for authentication module*' in '**Guidelines for Developers**' section for more details.

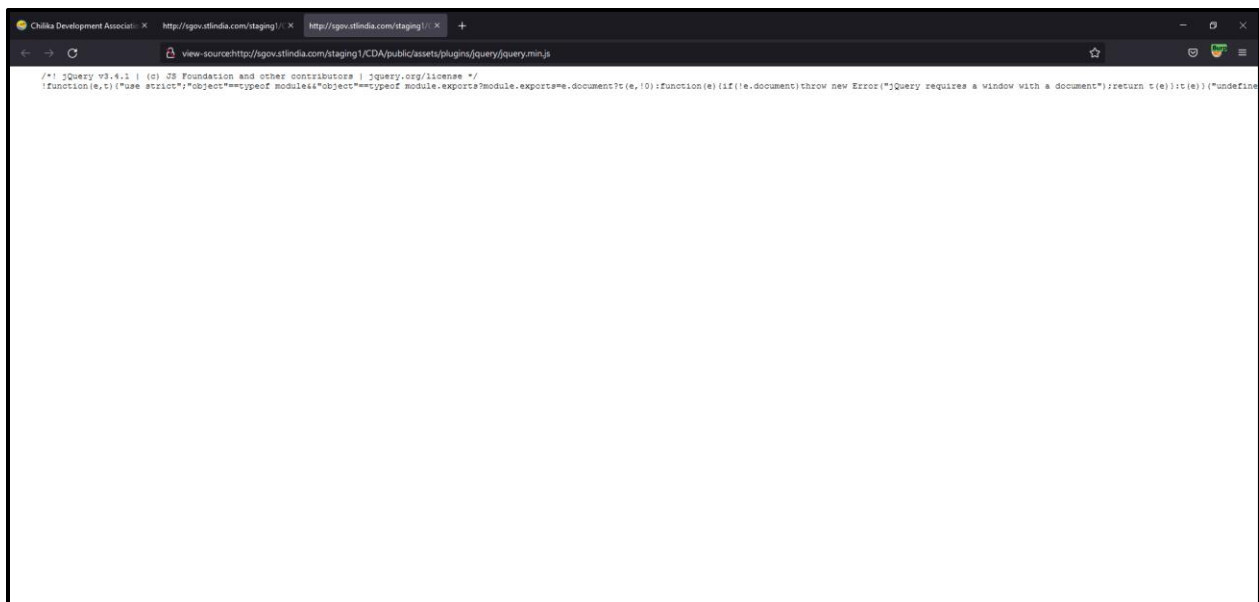
2. The application is using outdated known vulnerable components – Using Component with known vulnerabilities.

| Observation Details and Impact                                                                                                                        | Technical Finding Name                     | Risk Rating | Impact on Application | Probability of Attack |
|-------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------|-------------|-----------------------|-----------------------|
| The application is using vulnerable version of framework in the client-side, by which an attacker can possibly exploit already known vulnerabilities. | Using Component with known vulnerabilities | Low         | Low                   | Low                   |

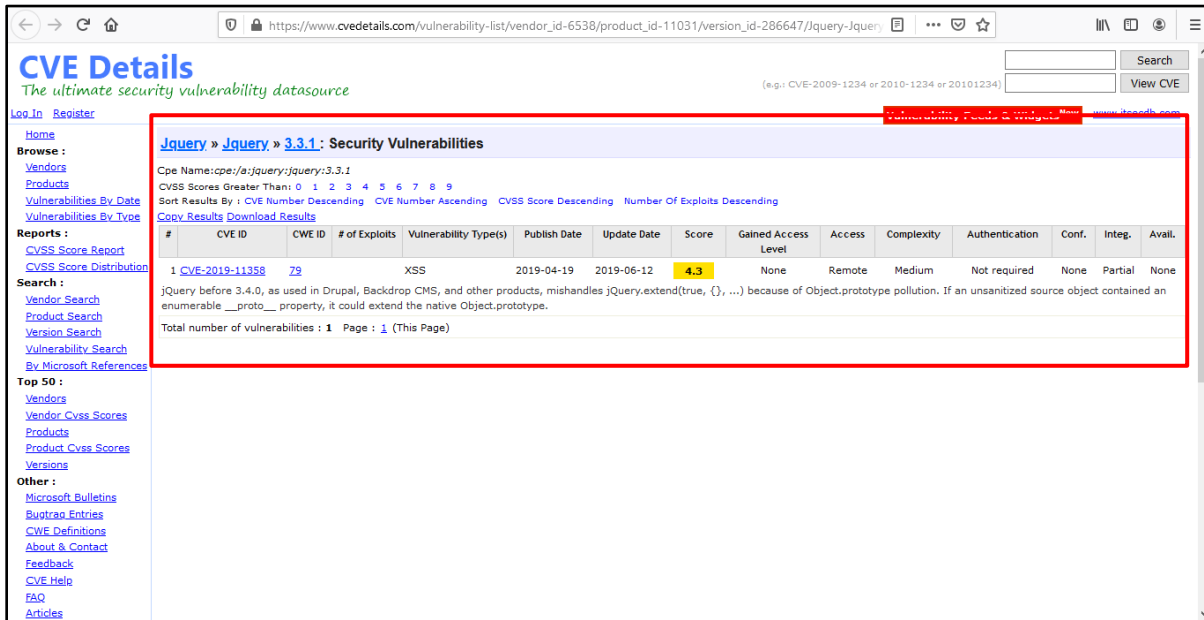
**How to test:**

**Instance #1: jQuery**

**Step #1:** The attacker also observed that the application discloses the jQuery and its version as shown below:



**Step #2:** The CVE details is as follows.



The screenshot shows the CVE Details website with a search for "jQuery » jQuery » 3.3.1: Security Vulnerabilities". The results table is as follows:

| # | CVE ID                         | CWE ID | # of Exploits | Vulnerability Type(s) | Publish Date | Update Date | Score | Gained Access Level | Access | Complexity | Authentication | Conf. | Integ.  | Avail. |
|---|--------------------------------|--------|---------------|-----------------------|--------------|-------------|-------|---------------------|--------|------------|----------------|-------|---------|--------|
| 1 | <a href="#">CVE-2019-11358</a> | 79     |               | XSS                   | 2019-04-19   | 2019-06-12  | 4.3   | None                | Remote | Medium     | Not required   | None  | Partial | None   |

The description for CVE-2019-11358 states: "jQuery before 3.4.0, as used in Drupal,Backdrop CMS, and other products, mishandles jQuery.extend(true, {}, ...) because of Object.prototype pollution. If an unsanitized source object contained an enumerable \_\_proto\_\_ property, it could extend the native Object.prototype."

Total number of vulnerabilities : 1 Page : 1 (This Page)

**Recommendation:**

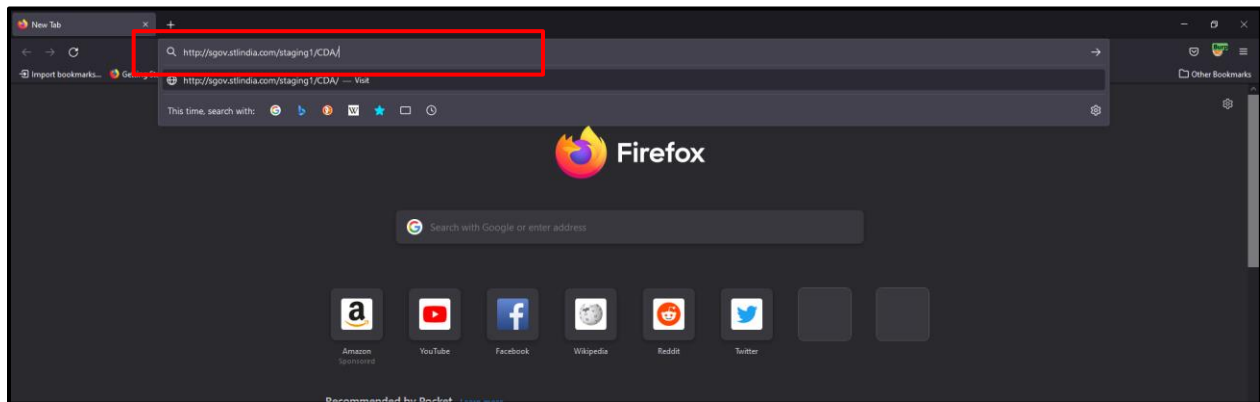
- The application must use the latest stable version of all the components or patch and update the components on regular basis.

**3. The application has not implemented 'Domain' cookie attribute – Insecure Cookie attributes.**

| Observation Details and Impact                                                                                                                                                                 | Technical Finding Name     | Risk Rating | Impact on Application | Probability of Attack |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------|-------------|-----------------------|-----------------------|
| The application has not implemented 'Domain' cookie attribute. The application cookie may get compromised by the attacker if these cookie attributes are not set properly for the application. | Insecure Cookie Attributes | Low         | Low                   | Low                   |

**How to Test:**

**Step #1:** An attacker navigates to the login page of the application at URL: <http://sgov.stlindia.com/staging1/CDA/> as shown below:





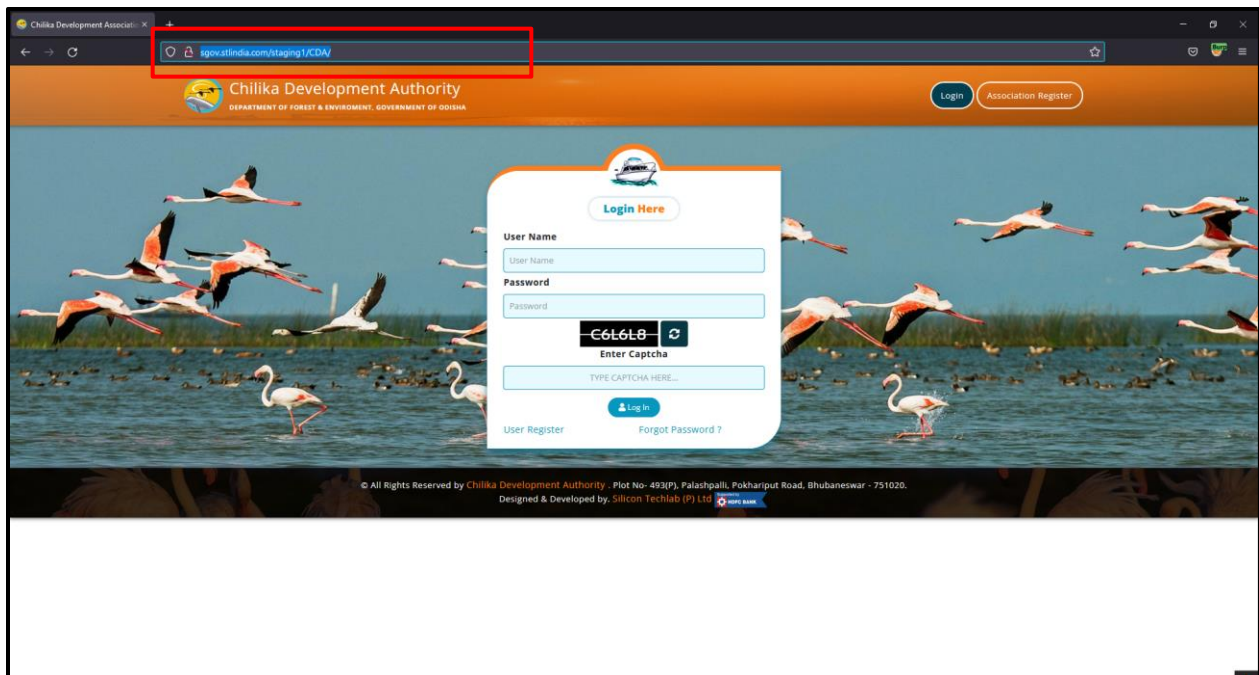


4. An application has enabled copy and paste feature on password fields of the 'Login Page' -  
Clipboard Data Stealing Attack.

| Observation Details and Impact                                                                                                                                                                                            | Technical Finding Name         | Risk Rating | Impact on Application | Probability of Attack |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------|-------------|-----------------------|-----------------------|
| The application has enabled copy and paste feature on the password field of 'Login page' page as a result a malicious user can steal the password as copy paste option is enabled thus compromising the victim's account. | Clipboard Data Stealing Attack | Low         | Low                   | Low                   |

**How to Test:**

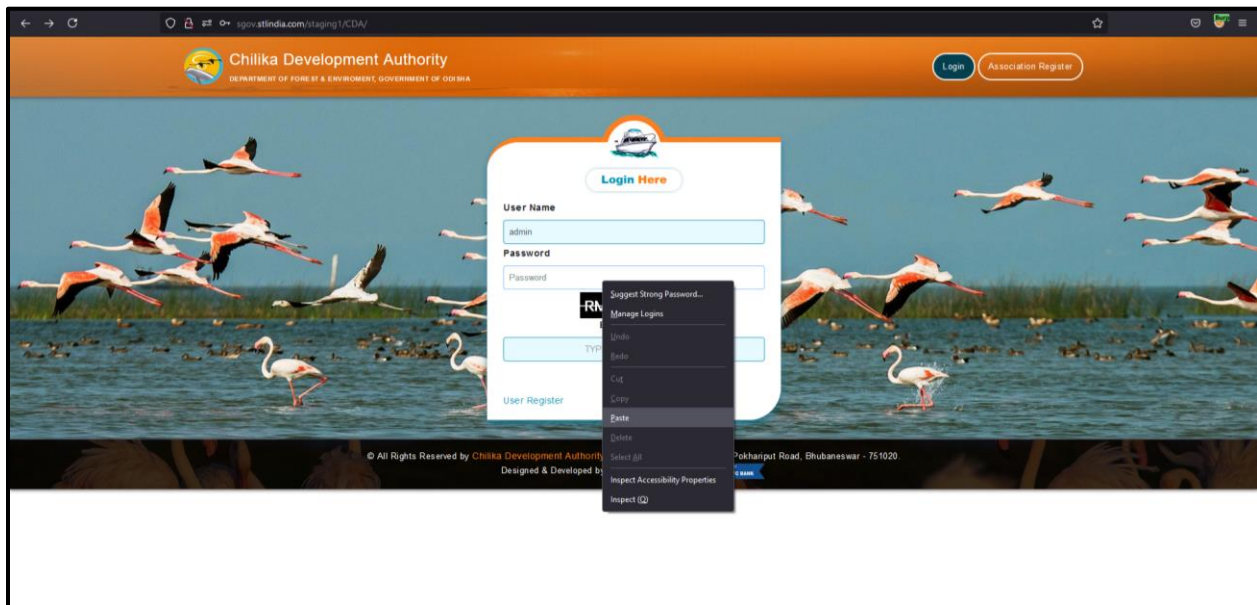
**Step #1:** A malicious user navigates to the login page of the application at the URL: <http://sgov.stlindia.com/staging1/CDA/> as shown below:



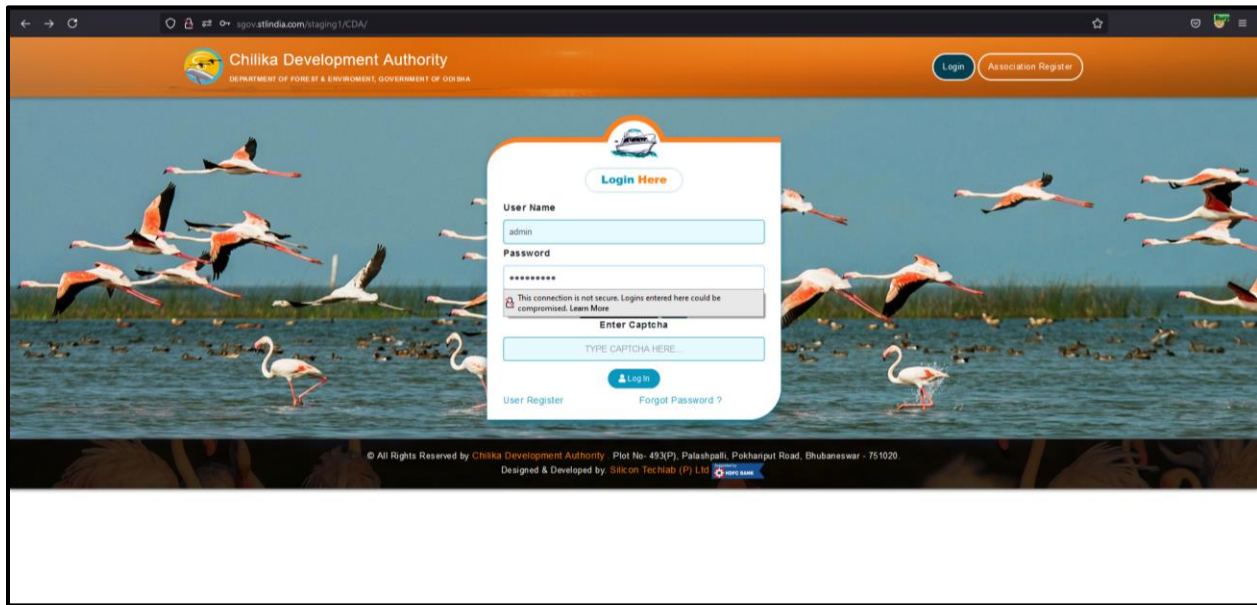
**Step #2:** Then the malicious user copies the password from a notepad as shown below:



**Step #3:** Then the malicious user navigates to the login page of the application as copy paste options is available malicious user paste the password as shown below:



**Step #4:** It can be observed that malicious user successfully paste the password as shown below:



**Recommendation:**

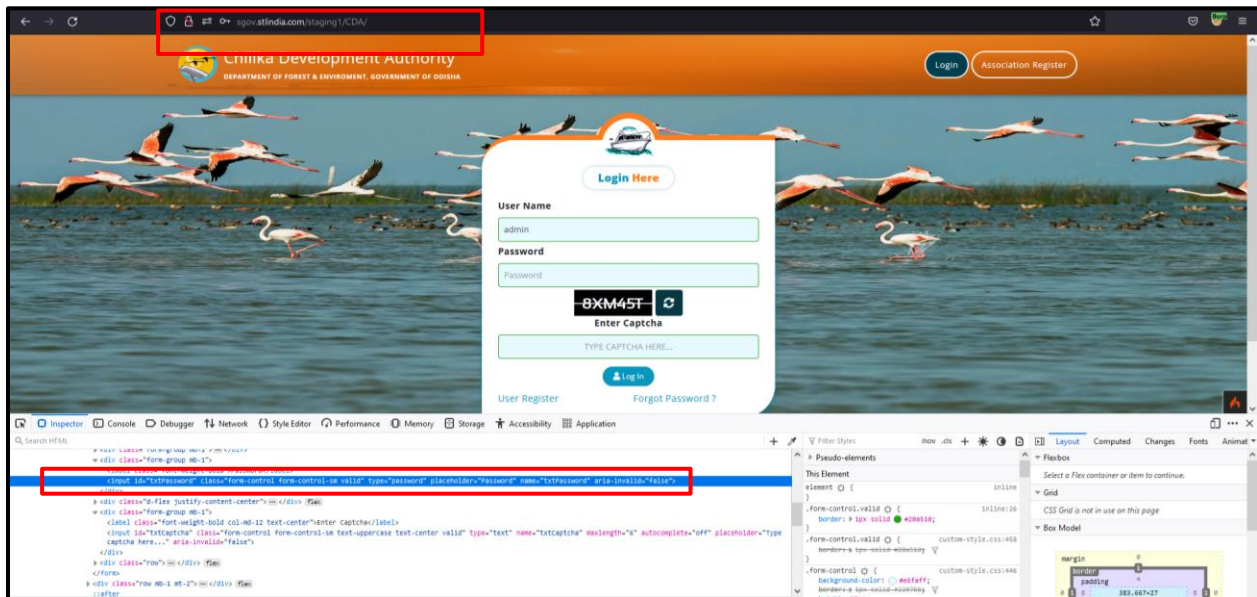
- The application should disable the '**copy**', '**paste**' option including the keyboard shortcuts of the same for the password field and for the other critical data fields.

5. An attacker can steal user data by using the advantage of the AUTOCOMPLETE browser feature –  
Autocomplete Issue.

| Observation Details and Impact                                                                                                                                                                | Technical Finding Name | Risk Rating | Impact on Application | Probability of Attack |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|-------------|-----------------------|-----------------------|
| The application has enabled the Autocomplete feature. An attacker can take advantage of the Autocomplete browser feature and can steal username, passwords of valid users of the application. | Autocomplete Issue     | Low         | Low                   | Low                   |

**How to test:**

**Step #1:** A customer navigates to the “Login” page of the application and observes that the autocomplete feature is not mentioned for the username and password as shown below:



**Recommendations:**

It is recommended to programmatically turn the AUTOCOMPLETE functionality to OFF by setting a HTML attribute on the page from the server side.

**Example:**

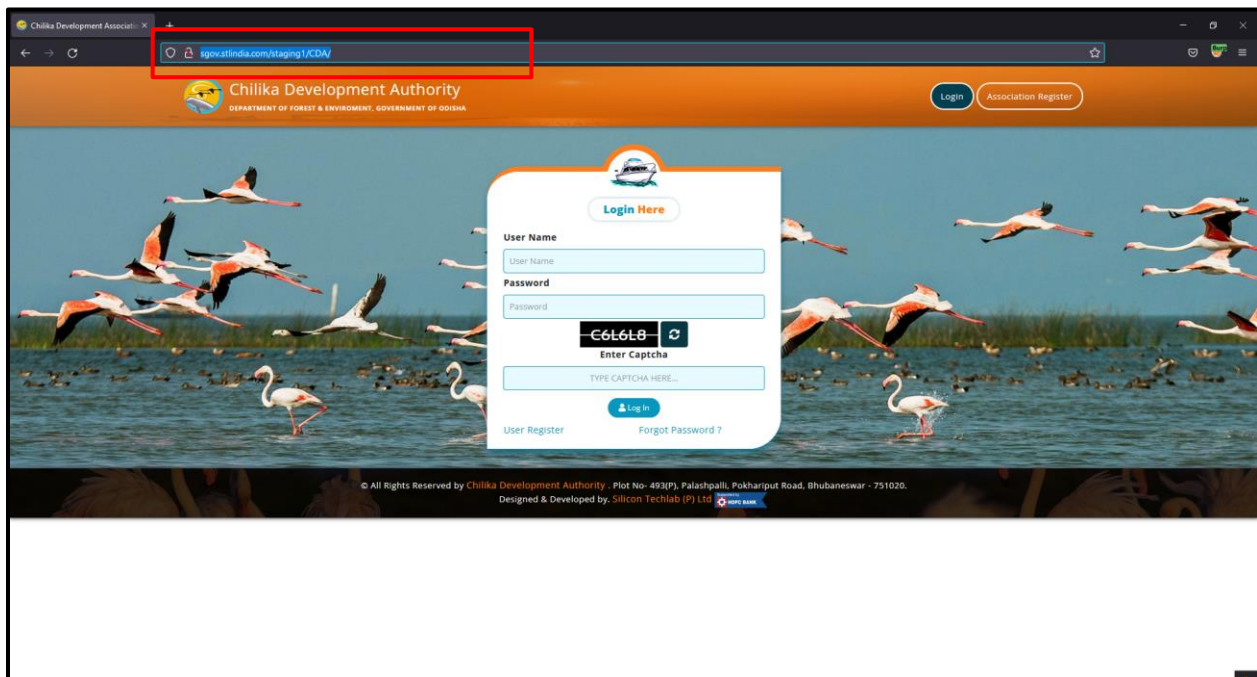
```
<form>
    .....
    .... other fields .....
    <input type="password" name="password" AUTOCOMPLETE="OFF">
    .....
</form>
```

6. An attacker may launch clickjacking attack against an application user - Clickjacking Attack.

| Observation Details and Impact                                                                                                                                                            | Technical Finding Name | Risk Rating | Impact on Application | Probability of Attack |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|-------------|-----------------------|-----------------------|
| An attacker can launch clickjacking attack against application user and can trick an application user to perform different activities allowed in the application without his/her consent. | Clickjacking Attack    | Low         | Low                   | Low                   |

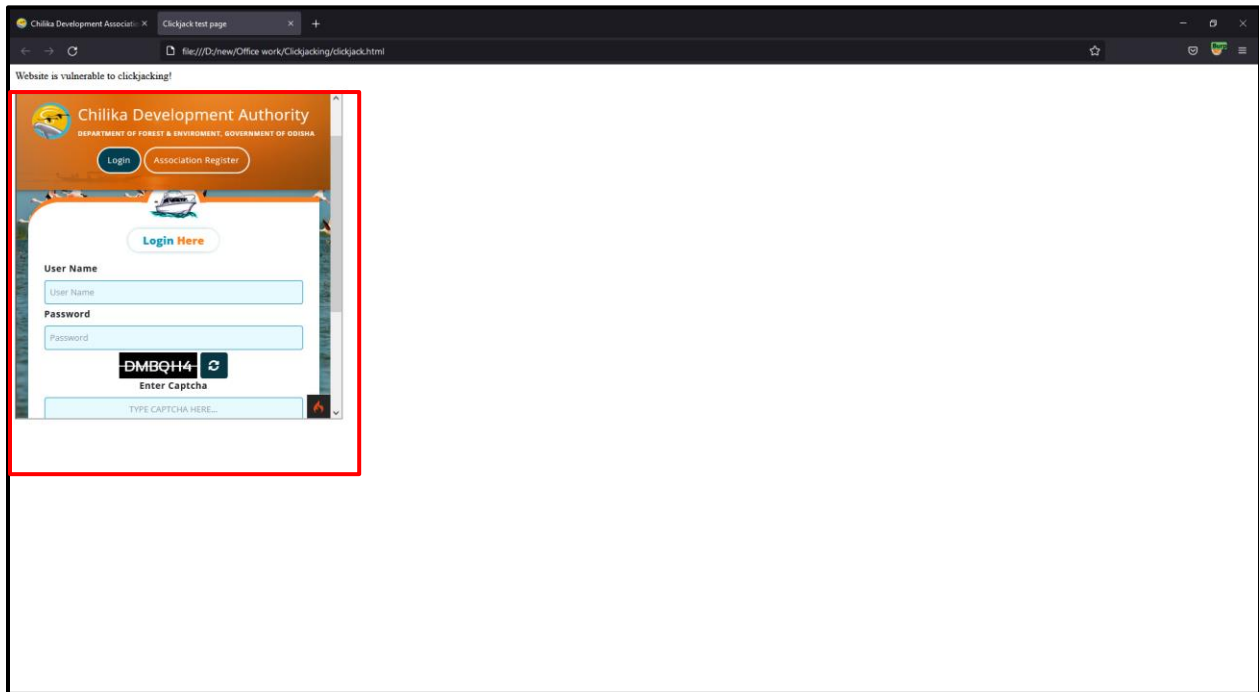
**How to Test:**

**Step #1:** A malicious user navigates to the login page of the application at the URL: <http://sgov.stlindia.com/staging1/CDA/> as shown below:

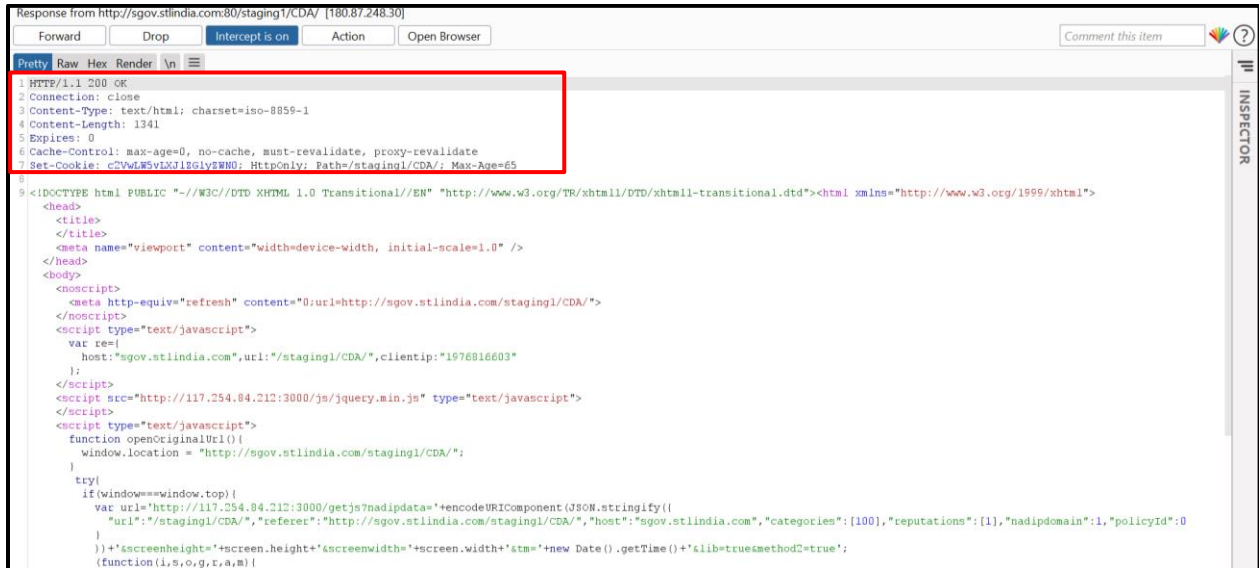




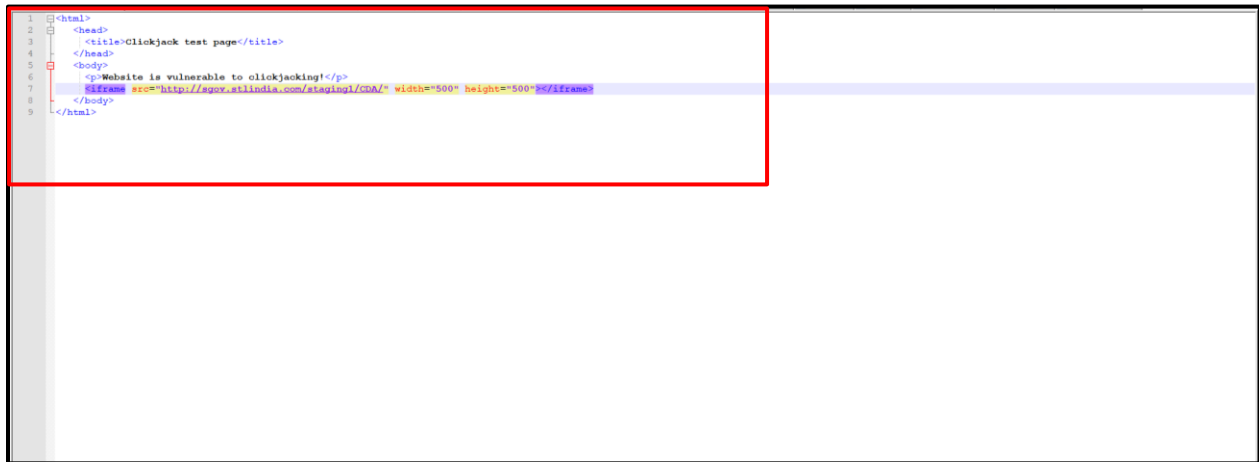
**Step #2:** The attacker framed the “Login” page of the application inside an HTML page as shown below:



**Step #3:** The attacker captures the response using an HTTP interceptor and observes that ‘X-Frame-Option=DENY’ is not properly implemented.



**Step #4:** The request for above page is triggered because of the iframe source as shown below:



```
1 <html>
2 <head>
3   <title>Clickjack test page</title>
4 </head>
5 <body>
6   <p>Website is vulnerable to clickjacking!</p>
7   <iframe src="http://agov.slindia.com/staging/CGA/" width="500" height="500"></iframe>
8 </body>
9 </html>
```

### **Recommendations:**

- The X-Frame-Options HTTP response header can be used to indicate whether a browser should be allowed to render a page in a <frame> or <iframe>. Sites can use this to avoid clickjacking attacks, by ensuring that their content is not embedded into other sites. Other way of mitigating click-jacking vulnerability is by implementing Frame busting code in the application which checks for framing page and the page being framed and allows the application to frame the page only if both resources are of same domain.

Possible values for the X-Frame-Options header are:

- X-Frame-Option = DENY must be implemented properly.
- SAMEORIGIN, which only allows the current site to frame the content.

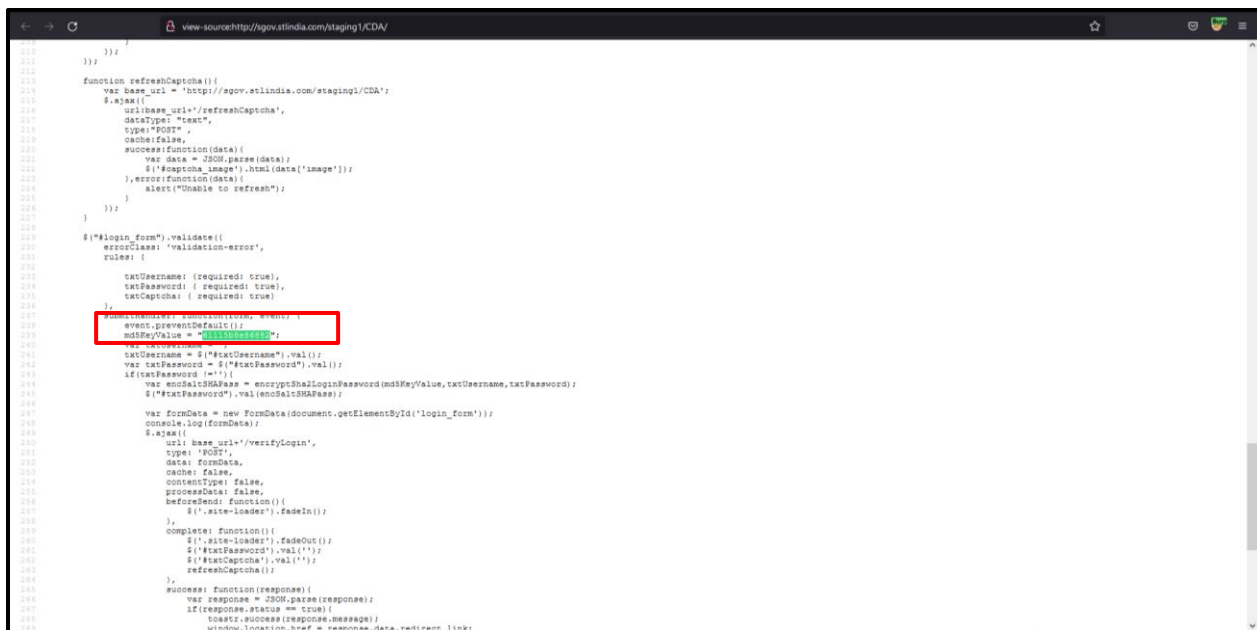


**7. The application is stores the static MD5 key value in text format in page source - Static md5 Key Used.**

| Observation Details and Impact                                                                                                                                            | Technical Finding Name | Risk Rating | Impact on Application | Probability of Attack |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|-------------|-----------------------|-----------------------|
| The application is stores the static MD5 key value in text format in page source which may allow an attacker carry out specific targeted attacks against the application. | Static md5 Key Used    | Low         | Low                   | Low                   |

**How to Test:**

**Step #1:** An attacker navigates to the page source of the application and it can be observed that the application is using static md5 key as shown below:



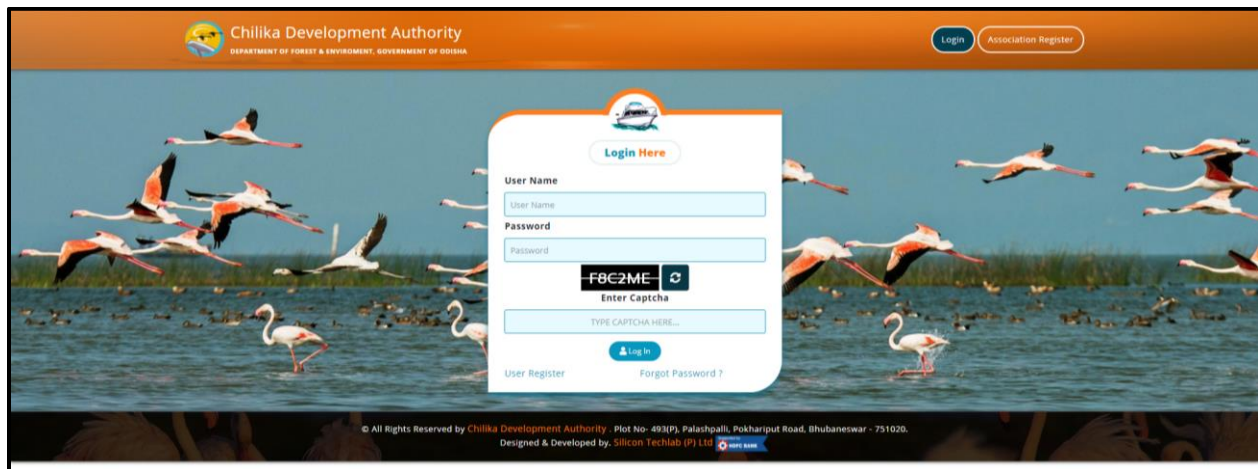
```
110  
111 }  
112 }  
113  
114 function refreshCaptcha() {  
115     var base_url = 'http://gov.atlindia.com/staging1/CDA';  
116     $.ajax({  
117         url: base_url + '/refreshCaptcha',  
118         dataType: 'text',  
119         type: 'POST',  
120         cache: false,  
121         success: function(data) {  
122             var data = JSON.parse(data);  
123             $('#captcha_image').html(data['image']);  
124             }, error: function(data) {  
125                 alert("Unable to refresh");  
126             }  
127         }  
128     });  
129 }  
130  
131 $('#login_form').validate({  
132     errorClass: 'validation-error',  
133     rules: {  
134         txtUsername: { required: true },  
135         txtPassword: { required: true },  
136         txtCaptcha: { required: true },  
137     },  
138     messages: {  
139         txtUsername: "Please enter a valid username",  
140         txtPassword: "Please enter a valid password",  
141         txtCaptcha: "Please enter a valid captcha",  
142     },  
143     submitHandler: function(form) {  
144         event.preventDefault();  
145         md5KeyValue = "5d41402eea408a71b384b6854879749e";  
146         var txtUsername = $('#txtUsername').val();  
147         var txtPassword = $('#txtPassword').val();  
148         if (txtPassword != '') {  
149             var encryptedPassword = encryptSHA1LoginPassword(md5KeyValue, txtUsername, txtPassword);  
150             $('#txtPassword').val(encryptedPassword);  
151         }  
152         var formData = new FormData(document.getElementById('login_form'));  
153         console.log(formData);  
154         $.ajax({  
155             url: base_url + '/verifyLogin',  
156             type: 'POST',  
157             data: formData,  
158             cache: false,  
159             contentType: false,  
160             processData: false,  
161             beforeSend: function() {  
162                 $('#site-loader').fadeIn();  
163             },  
164             complete: function() {  
165                 $('#site-loader').fadeOut();  
166                 $('#txtUsername').val('');  
167                 $('#txtPassword').val('');  
168                 $('#txtCaptcha').val('');  
169                 refreshCaptcha();  
170             },  
171             success: function(response) {  
172                 var response = JSON.parse(response);  
173                 if (response.status == true) {  
174                     showAlert(response.message);  
175                     window.location.href = response.data.redirect_link;  
176                 }  
177             }  
178         });  
179     }  
180 }  
181 }  
182 }  
183 }  
184 }  
185 }  
186 }  
187 }  
188 }  
189 }  
190 }  
191 }  
192 }  
193 }  
194 }  
195 }  
196 }  
197 }  
198 }  
199 }  
200 }  
201 }  
202 }  
203 }  
204 }  
205 }  
206 }  
207 }  
208 }  
209 }  
210 }  
211 }  
212 }  
213 }  
214 }  
215 }  
216 }  
217 }  
218 }  
219 }  
220 }  
221 }  
222 }  
223 }  
224 }  
225 }  
226 }  
227 }  
228 }  
229 }  
230 }  
231 }  
232 }  
233 }  
234 }  
235 }  
236 }  
237 }  
238 }  
239 }  
240 }  
241 }  
242 }  
243 }  
244 }  
245 }  
246 }  
247 }  
248 }  
249 }  
250 }  
251 }  
252 }  
253 }  
254 }  
255 }  
256 }  
257 }  
258 }  
259 }  
260 }  
261 }  
262 }  
263 }  
264 }  
265 }  
266 }  
267 }  
268 }  
269 }  
270 }  
271 }  
272 }  
273 }  
274 }  
275 }  
276 }  
277 }  
278 }  
279 }  
280 }  
281 }  
282 }  
283 }  
284 }  
285 }  
286 }  
287 }  
288 }  
289 }  
290 }  
291 }  
292 }  
293 }  
294 }  
295 }  
296 }  
297 }  
298 }  
299 }  
300 }  
301 }  
302 }  
303 }  
304 }  
305 }  
306 }  
307 }  
308 }  
309 }  
310 }  
311 }  
312 }  
313 }  
314 }  
315 }  
316 }  
317 }  
318 }  
319 }  
320 }  
321 }  
322 }  
323 }  
324 }  
325 }  
326 }  
327 }  
328 }  
329 }  
330 }  
331 }  
332 }  
333 }  
334 }  
335 }  
336 }  
337 }  
338 }  
339 }  
340 }  
341 }  
342 }  
343 }  
344 }  
345 }  
346 }  
347 }  
348 }  
349 }  
350 }  
351 }  
352 }  
353 }  
354 }  
355 }  
356 }  
357 }  
358 }  
359 }  
360 }  
361 }  
362 }  
363 }  
364 }  
365 }  
366 }  
367 }  
368 }  
369 }  
370 }  
371 }  
372 }  
373 }  
374 }  
375 }  
376 }  
377 }  
378 }  
379 }  
380 }  
381 }  
382 }  
383 }  
384 }  
385 }  
386 }  
387 }  
388 }  
389 }  
390 }  
391 }  
392 }  
393 }  
394 }  
395 }  
396 }  
397 }  
398 }  
399 }  
400 }  
401 }  
402 }  
403 }  
404 }  
405 }  
406 }  
407 }  
408 }  
409 }  
410 }  
411 }  
412 }  
413 }  
414 }  
415 }  
416 }  
417 }  
418 }  
419 }  
420 }  
421 }  
422 }  
423 }  
424 }  
425 }  
426 }  
427 }  
428 }  
429 }  
430 }  
431 }  
432 }  
433 }  
434 }  
435 }  
436 }  
437 }  
438 }  
439 }  
440 }  
441 }  
442 }  
443 }  
444 }  
445 }  
446 }  
447 }  
448 }  
449 }  
450 }  
451 }  
452 }  
453 }  
454 }  
455 }  
456 }  
457 }  
458 }  
459 }  
460 }  
461 }  
462 }  
463 }  
464 }  
465 }  
466 }  
467 }  
468 }  
469 }  
470 }  
471 }  
472 }  
473 }  
474 }  
475 }  
476 }  
477 }  
478 }  
479 }  
480 }  
481 }  
482 }  
483 }  
484 }  
485 }  
486 }  
487 }  
488 }  
489 }  
490 }  
491 }  
492 }  
493 }  
494 }  
495 }  
496 }  
497 }  
498 }  
499 }  
500 }  
501 }  
502 }  
503 }  
504 }  
505 }  
506 }  
507 }  
508 }  
509 }  
510 }  
511 }  
512 }  
513 }  
514 }  
515 }  
516 }  
517 }  
518 }  
519 }  
520 }  
521 }  
522 }  
523 }  
524 }  
525 }  
526 }  
527 }  
528 }  
529 }  
530 }  
531 }  
532 }  
533 }  
534 }  
535 }  
536 }  
537 }  
538 }  
539 }  
540 }  
541 }  
542 }  
543 }  
544 }  
545 }  
546 }  
547 }  
548 }  
549 }  
550 }  
551 }  
552 }  
553 }  
554 }  
555 }  
556 }  
557 }  
558 }  
559 }  
560 }  
561 }  
562 }  
563 }  
564 }  
565 }  
566 }  
567 }  
568 }  
569 }  
570 }  
571 }  
572 }  
573 }  
574 }  
575 }  
576 }  
577 }  
578 }  
579 }  
580 }  
581 }  
582 }  
583 }  
584 }  
585 }  
586 }  
587 }  
588 }  
589 }  
590 }  
591 }  
592 }  
593 }  
594 }  
595 }  
596 }  
597 }  
598 }  
599 }  
600 }  
601 }  
602 }  
603 }  
604 }  
605 }  
606 }  
607 }  
608 }  
609 }  
610 }  
611 }  
612 }  
613 }  
614 }  
615 }  
616 }  
617 }  
618 }  
619 }  
620 }  
621 }  
622 }  
623 }  
624 }  
625 }  
626 }  
627 }  
628 }  
629 }  
630 }  
631 }  
632 }  
633 }  
634 }  
635 }  
636 }  
637 }  
638 }  
639 }  
640 }  
641 }  
642 }  
643 }  
644 }  
645 }  
646 }  
647 }  
648 }  
649 }  
650 }  
651 }  
652 }  
653 }  
654 }  
655 }  
656 }  
657 }  
658 }  
659 }  
660 }  
661 }  
662 }  
663 }  
664 }  
665 }  
666 }  
667 }  
668 }  
669 }  
670 }  
671 }  
672 }  
673 }  
674 }  
675 }  
676 }  
677 }  
678 }  
679 }  
680 }  
681 }  
682 }  
683 }  
684 }  
685 }  
686 }  
687 }  
688 }  
689 }  
690 }  
691 }  
692 }  
693 }  
694 }  
695 }  
696 }  
697 }  
698 }  
699 }  
700 }  
701 }  
702 }  
703 }  
704 }  
705 }  
706 }  
707 }  
708 }  
709 }  
710 }  
711 }  
712 }  
713 }  
714 }  
715 }  
716 }  
717 }  
718 }  
719 }  
720 }  
721 }  
722 }  
723 }  
724 }  
725 }  
726 }  
727 }  
728 }  
729 }  
730 }  
731 }  
732 }  
733 }  
734 }  
735 }  
736 }  
737 }  
738 }  
739 }  
740 }  
741 }  
742 }  
743 }  
744 }  
745 }  
746 }  
747 }  
748 }  
749 }  
750 }  
751 }  
752 }  
753 }  
754 }  
755 }  
756 }  
757 }  
758 }  
759 }  
760 }  
761 }  
762 }  
763 }  
764 }  
765 }  
766 }  
767 }  
768 }  
769 }  
770 }  
771 }  
772 }  
773 }  
774 }  
775 }  
776 }  
777 }  
778 }  
779 }  
780 }  
781 }  
782 }  
783 }  
784 }  
785 }  
786 }  
787 }  
788 }  
789 }  
790 }  
791 }  
792 }  
793 }  
794 }  
795 }  
796 }  
797 }  
798 }  
799 }  
800 }  
801 }  
802 }  
803 }  
804 }  
805 }  
806 }  
807 }  
808 }  
809 }  
810 }  
811 }  
812 }  
813 }  
814 }  
815 }  
816 }  
817 }  
818 }  
819 }  
820 }  
821 }  
822 }  
823 }  
824 }  
825 }  
826 }  
827 }  
828 }  
829 }  
830 }  
831 }  
832 }  
833 }  
834 }  
835 }  
836 }  
837 }  
838 }  
839 }  
840 }  
841 }  
842 }  
843 }  
844 }  
845 }  
846 }  
847 }  
848 }  
849 }  
850 }  
851 }  
852 }  
853 }  
854 }  
855 }  
856 }  
857 }  
858 }  
859 }  
860 }  
861 }  
862 }  
863 }  
864 }  
865 }  
866 }  
867 }  
868 }  
869 }  
870 }  
871 }  
872 }  
873 }  
874 }  
875 }  
876 }  
877 }  
878 }  
879 }  
880 }  
881 }  
882 }  
883 }  
884 }  
885 }  
886 }  
887 }  
888 }  
889 }  
890 }  
891 }  
892 }  
893 }  
894 }  
895 }  
896 }  
897 }  
898 }  
899 }  
900 }  
901 }  
902 }  
903 }  
904 }  
905 }  
906 }  
907 }  
908 }  
909 }  
910 }  
911 }  
912 }  
913 }  
914 }  
915 }  
916 }  
917 }  
918 }  
919 }  
920 }  
921 }  
922 }  
923 }  
924 }  
925 }  
926 }  
927 }  
928 }  
929 }  
930 }  
931 }  
932 }  
933 }  
934 }  
935 }  
936 }  
937 }  
938 }  
939 }  
940 }  
941 }  
942 }  
943 }  
944 }  
945 }  
946 }  
947 }  
948 }  
949 }  
950 }  
951 }  
952 }  
953 }  
954 }  
955 }  
956 }  
957 }  
958 }  
959 }  
960 }  
961 }  
962 }  
963 }  
964 }  
965 }  
966 }  
967 }  
968 }  
969 }  
970 }  
971 }  
972 }  
973 }  
974 }  
975 }  
976 }  
977 }  
978 }  
979 }  
980 }  
981 }  
982 }  
983 }  
984 }  
985 }  
986 }  
987 }  
988 }  
989 }  
990 }  
991 }  
992 }  
993 }  
994 }  
995 }  
996 }  
997 }  
998 }  
999 }  
1000 }
```

**8. The captcha is not properly implemented in the application - CAPTCHA In response.**

| Observation Details and Impact                                                                                           | Technical Finding Name | Risk Rating | Impact on Application | Probability of Attack |
|--------------------------------------------------------------------------------------------------------------------------|------------------------|-------------|-----------------------|-----------------------|
| The captcha is not properly implemented in the application:<br>a) CAPTCHA is travelling in clear text format in response | CAPTCHA In response    | Low         | Low                   | Low                   |

**How to Test:**

**Step #1:** An attacker navigates to the login page of the application at URL: <http://sgov.stlindia.com/staging1/CDA/> as shown below:



**Step #2:** By capturing the response it was observed that the application sending the CAPTCHA value in clear text in response as shown below:



**Recommendation:**

- It is recommended to application should not sent the CAPTCHA value in the response in clear text format.

### 3. Appendix – 1: Recommended Action Plan

The recommended action items are prioritized in order of Severity. Each item is given a ranking of High, Medium, or Low. It is recommended that Action items with severity of High ranking should be addressed immediately, action items with Medium Severity tentatively within 1 month and action items with Low severity ranking within 3 months' time. Internal information security team may choose under their discretion to change the timeline to fix the issues as per organization's requirements. The following table shall be used in follow-up process to ensure the recommended controls are implemented and the application is secure.

| #  | Technical Finding Name                     | Severity | Responsibility | Due Date | Status |
|----|--------------------------------------------|----------|----------------|----------|--------|
| 1. | Insecure Transmission of Password          | High     |                |          |        |
| 2. | Using Component with known vulnerabilities | Low      |                |          |        |
| 3. | Insecure Cookie Attributes                 | Low      |                |          |        |
| 4. | Clipboard Data Stealing Attack             | Low      |                |          |        |
| 5. | Autocomplete Issue                         | Low      |                |          |        |
| 6. | Clickjacking Attack                        | Low      |                |          |        |
| 7. | Static md5 Key Used                        | Low      |                |          |        |
| 8. | CAPTCHA In response                        | Low      |                |          |        |

## 4. Appendix – 2: Guidelines for Developers

### 4.1 Salted SHA256 technique for the authentication module

Here is how salted SHA256 technique works:

1. With each request to the login page, the server generates a random number, the salt, and sends it to the client along with the page.
2. A JavaScript code on the client computes the SHA256 hash of the password entered by the user.
3. The salt is concatenated with the computed SHA256 of the password to re-compute the SHA256 hash of the concatenated string (let this SHA256 hash be A).
4. The result is sent to the server along with the user-id.
5. The server picks the SHA256 of the password for the corresponding user-id from the database.
6. The SHA256 password fetched from the database is concatenated with the salt stored at the server which was sent to the client. SHA256 of the concatenated string is calculated at the server (let this SHA256 hash be B).
7. If the user entered the correct password, then these two hashes (A & B) should match. The server compares these two hashes and if they match, the user is authenticated.

## 5. Appendix – 3: Interpretation of Ratings

The Severity for each finding in this report is based on the Impact and Ease of exploitation of the vulnerability. Here's a guide to interpreting the Severity:

### High Risk:

These are vulnerabilities that can be exploited easily and have high impact when exploited. These issues pose a significant threat to security and have high possibility on the extent to which they can be abused. The issues that have a high risk are the ones that allow an attacker to gain privileged access to the system, create DoS in a critical service or allow significant modification of configuration to affect the system security would fall into this category.

### Medium Risk:

These are vulnerabilities that do not pose an immediate risk to your system but may do so over time. Vulnerabilities that require other conditions to be fulfilled and by themselves alone do not pose a direct risk to the system would be covered in this category. Vulnerabilities for which no publicly known exploit exists are usually rated as medium risk. We recommend that you attend to these medium risk vulnerabilities as soon as you have fixed the high-risk ones. It may be noted that today's medium risk vulnerabilities might become high risk tomorrow.

### Low Risk:

Three types of vulnerabilities get this rating: first, vulnerabilities that can only be exploited locally, secondly, vulnerabilities that are easy to exploit but have a low impact, and finally vulnerabilities that reveal information that might aid an attacker in crafting an attack easily. Descriptive error or informational messages are an example of this type. Clear text protocols being enabled and unnecessary services running on the target system would typically come under this category. Resumes of employees that discuss internal architecture and features they have configured on security devices are another example. Please fix them before the next testing cycle.

## 6. Appendix – 4: Tools Used

The following is a list of tools that were used during the grey box application security audit activity.

1. **Burp Suite:** Burp Proxy is a tool which intercepts traffic between client and server. It is available for download at <http://www.portswigger.net/>
2. **Directory Buster**

END OF DOCUMENT